

BBM460

Proposal

Report

Group Members:

Orhun İnan, Rıza Çakır

Project Title:

P2P Fault Detection Protocol with Decentralized Reporting

Project Summary

Industrial environments such as chemical, petroleum, gas, and oil facilities are highly susceptible to critical faults like LPG leakage and unexpected fires. Timely detection and reporting of these hazards are vital for ensuring worker safety and preventing catastrophic damage. This project proposes an Internet of Things (IoT)-based fault monitoring system designed to detect gas leaks and fires.

Unlike traditional centralized systems where a single remote server failure can compromise the entire safety network, this project introduces a decentralized architecture at the edge. Utilizing ESP32 microcontrollers for sensor nodes and the lightweight MQTT protocol, devices will communicate directly with their peers on the local network. This ensures that fault detection and localized alerts remain functional even if external internet connectivity is lost. Furthermore, to guarantee uninterrupted external communication, the system employs multiple Single Board Computers (SBCs) acting as redundant reporting nodes. Each of these SBCs functions as a "mini central node." If one reporting node fails, the others seamlessly continue to log data, bridge to external networks, and serve the web-based dashboard, ensuring that remote personnel are always informed in real-time.

Method

The system will be built upon a local mesh of independent ESP32 nodes and a redundant cluster of SBC reporting gateways.

1. **Sensory Data Collection at the Edge:** Each ESP32 node will be equipped with sensors to continuously monitor its immediate industrial environment.
2. **Decentralized Communication:** The nodes will utilize the MQTT protocol over Wi-Fi to publish and subscribe to specific fault topics.
3. **P2P Alerting Logic:** If a node detects a gas leak, it publishes an emergency payload to a localized MQTT topic. Neighboring ESP32 nodes subscribed to this topic will immediately receive the message and trigger local physical alarms. The logic is distributed across the nodes, relying on peer communication rather than a master controller.
4. **Redundant Local Reporting & Bridging:** Linux-based SBCs on the same network will act as reporting nodes. They will listen to the fault topics and act as high-availability gateways. If one SBC loses power or its external internet connection, the other active SBCs will continue pushing critical alerts to administrators.

Technologies to be Used

- **Hardware:**
 - **Sensor Nodes:** ESP32 microcontrollers.
 - **Reporting Nodes:** Raspberry Pi SBC
 - **Sensors & Actuators:** Gas Sensors, Flame sensors, Buzzers, and LEDs.
- **Software & Protocols:**
 - **Communication:** MQTT Protocol.
 - **Node Firmware:** C/C++ for hardware-level control on the ESP32s.

- **Reporting Backend & Dashboard:** A high-performance backend service (e.g., written in Go or C# .NET) running on the SBCs to handle concurrent MQTT logging and serve the real-time monitoring dashboard to clients.

Project Plan (Timeline):

Phase	Duration	Tasks / Milestones
Phase 1	Weeks 1-2	Requirement analysis, hardware procurement, and setting up the cross-compilation and development environments.
Phase 2	Weeks 3-4	Component testing: Interfacing sensors with the ESP32s and writing the C++ data-reading routines.
Phase 3	Weeks 5-6	Network setup: Configuring the Linux SBC network, deploying the MQTT brokers, and verifying ESP32 connections.
Phase 4	Weeks 7-8	Implementing the P2P logic: Programming the ESP32s to publish fault states and trigger local hardware alarms based on peer messages.
Phase 5	Weeks 9-10	Backend & Web integration: Developing the redundant backend services on the SBCs to log payloads and serve the monitoring dashboard.
Phase 6	Weeks 11-12	System integration, simulating SBC node failures to test high-availability and P2P resilience, debugging, and final report documentation.